UNIVERSITY
OF OSLO

**Project Report**

# Quantum Machine Learning for Binary Classification

A Comparative Study of Data Encoders and Ansatz Circuits on the Iris Dataset

**Javid Rezai**
**javid.rezai@uio.no**

Department of Informatics
Faculty of Mathematics and Natural Sciences


6th June 2025

**Abstract**

Quantum machine learning (QML) represents a promising intersection of quantum computing and classical machine learning, leveraging the exponential scaling of quantum Hilbert spaces to potentially achieve computational advantages. Addressing the empirical challenges in designing quantum variational classifiers, which parallel those in classical neural networks, this report presents a comprehensive study of their application to binary classification on the Iris dataset. We systematically investigate the impact of various architectural choices and training parameters. This includes comparing two distinct quantum data encoders (a simple angle encoder and an advanced ZZ feature map), evaluating parameterized ansatz circuits of varying complexity and depth (number of layers), and examining the influence of optimization hyperparameters such as batch size and learning rate for the Adam optimizer. Our quantum classifiers are trained using the parameter-shift rule for analytical gradient computation. Experimental results demonstrate that specific configurations, such as an advanced encoder with ZZ interactions, can achieve high classification performance, reaching even up to 100% accuracy on the test set. More broadly, this study provides insights into the sensitivity of QML models to these design choices, mirroring the hyperparameter tuning process common in deep learning. These findings contribute to the growing understanding of QML capabilities and limitations, offering guidance for designing effective models in the near-term quantum computing era.

For full code implementation and results refer to GitHub.[1]

[1] https://github.com/javidaf/quantum-computing/tree/master/quantum_computing/p2

# Chapter 1

# Introduction

The intersection of quantum computing and machine learning has emerged as one of the most promising research directions in computational science. Quantum machine learning (QML) leverages the unique properties of quantum systems, particularly the exponential scaling of quantum Hilbert spaces, to potentially achieve computational advantages over classical approaches. The development and optimization of QML models, especially variational quantum algorithms, involve an empirical process of selecting data encodings, ansatz architectures, and training methodologies, akin to the design and tuning of classical neural networks.

The fundamental motivation for quantum machine learning stems from a striking observation about information capacity. While $n$ classical bits can store $n$ boolean variables, the quantum state of $n$ qubits requires $2^n$ complex amplitudes for complete description. This exponential scaling suggests that quantum systems might naturally represent and process high-dimensional data more efficiently than classical computers.

However, this exponential advantage comes with significant constraints. Parameterized quantum circuits (PQCs), which form the backbone of most QML algorithms, can only explore a polynomially small subset of the full Hilbert space due to practical limitations on circuit depth and gate count [1, Sec. 4.5.4]. Despite this limitation, the accessible quantum states may still be classically intractable to compute while remaining efficiently preparable on quantum hardware. The challenge lies in designing these PQCs and training them effectively, much like architecting and training deep neural networks.

## 1.1 Problem Statement and Objectives

This study addresses the fundamental question of how different quantum data encoding strategies, ansatz circuit designs (including structural complexity and depth), and optimization hyperparameters affect the performance of quantum classifiers. This investigation mirrors the empirical approach often taken in classical machine learning to identify optimal model configurations. Specifically, we investigate the comparative effectiveness of different quantum data encoding schemes, namely a simple angle encoding versus an advanced ZZ feature map. Furthermore, we explore the impact of ansatz circuit architecture, encompassing its complexity and the number of layers, on classification performance. The study also examines the influence of key optimization hyperparameters, such as batch size during training and the learning rate of the Adam optimizer, on convergence and final model accuracy. A core objective is the practical implementation of quantum machine learning using the parameter-shift rule for analytical gradient computation. Finally, we evaluate the performance of the

developed quantum classifiers on a well-established classical dataset (Iris, first two classes), including a systematic grid search over encoder types, ansatz variants, learning rates, batch sizes, and ansatz layers to identify the best-performing configuration.

We focus on binary classification using the first two classes of the Iris dataset, providing a controlled environment to evaluate these multifaceted aspects of quantum machine learning approaches and compare their performance.

## 1.2 Contributions

The main contributions of this work encompass the implementation of QML model with flexible architectural design, such that it can accommodate various data encoders, ansatzes, and optimization strategies, similar to the empirical design of classical neural networks. Further contributions include a systematic investigation of the impact of the different quantum data encoding strategies, ansatz circuit designs, and optimization hyperparameters on the performance of quantum classifiers.

## 1.3 Report Structure

This report is organized as follows: Chapter 2 provides the necessary background on quantum computing fundamentals and quantum machine learning concepts. Chapter 3 details our methodology, including the implementation of data encoders, ansatz circuits, and optimization procedures. Chapter 4 presents experimental results and discussion and performence anlysis. Chapter 5 concludes with a summary of key insights and directions for future research.

# Chapter 2

# Background and Theory

This chapter establishes the theoretical foundation for quantum machine learning, covering essential quantum computing concepts and the mathematical framework underlying variational quantum algorithms.

## 2.1 Quantum Computing Fundamentals

**Quantum States and Hilbert Spaces**

A quantum system with $n$ qubits exists in a $2^n$-dimensional complex Hilbert space $\mathcal{H}$. The state of the system is described by a unit vector:

$$|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle \tag{2.1}$$

where $\alpha_i \in \mathbb{C}$ are complex amplitudes satisfying the normalization condition $\sum_{i=0}^{2^n-1} |\alpha_i|^2 = 1$, and $\{|i\rangle\}$ forms the computational basis.

For a single qubit, the general state is:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \tag{2.2}$$

where $|\alpha|^2 + |\beta|^2 = 1.$ and $|0\rangle, |1\rangle$ are the basis states defined as:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \tag{2.3}$$

In our case working with two qbits we build the two-qubit system as:

$$|\psi\rangle = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle \tag{2.4}$$

where $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ are the basis states for two qubits defined as the tensor product of the individual single qubit states for example:

$$|00\rangle = |0\rangle \otimes |0\rangle = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \quad |01\rangle = |0\rangle \otimes |1\rangle = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \tag{2.5}$$

**Quantum Gates and Circuits**

Quantum gates are unitary operators that transform quantum states. Key gates used in this work include:

**Hadamard Gate:** Creates superposition states

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \tag{2.6}$$

**Rotation Gates:** Parameterized single-qubit rotations

$$R_z(\theta) = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix} \tag{2.7}$$

$$R_y(\theta) = \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix} \tag{2.8}$$

**CNOT Gate:** Two-qubit entangling gate with control and target qubits

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \tag{2.9}$$

**Quantum Circuits** are built by sequencially applying quantum gates (discussed above) to qibits. This prepares the quantum state for measurement.

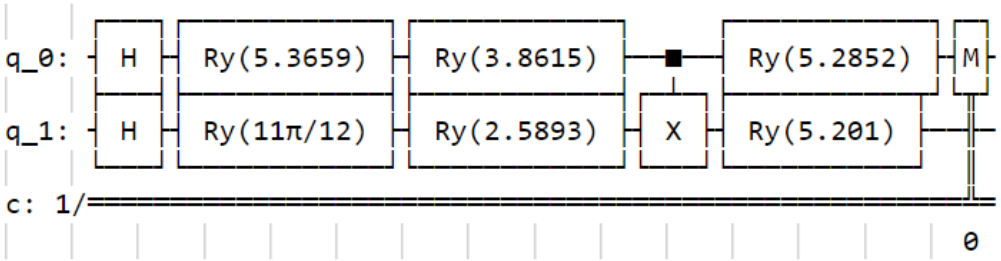


Figure 2.1: A two qubit quantum circuit that contains Hadamard gates (H), rotation gates (Ry($\theta$)), and a CNOT ($X$) gate. The rotation angles are random in this figure. At the end there is a measurement operation

The complete quantum circuit integrates data encoding, parameterized operations, and measurement to form a trainable model capable of pattern recognition. Figure 2.1 illustrates an example structure of our quantum circuit implementation.

## 2.2 Quantum Machine Learning Framework

### 2.2.1 Overview of Classical Machine Learning

Machine learning (ML) is a subfield of artificial intelligence focused on developing algorithms that enable computer systems to learn from and make decisions or predictions based on data, without being explicitly programmed for each task. The core idea is to identify patterns in data and use these patterns to generalize to new, unseen data.

ML tasks are broadly categorized. In **supervised learning**, the algorithm learns from a labeled dataset, where each data point consists of input features and a corresponding known output or target label. The goal is to learn a mapping function that can predict the output for new input features. Classification (predicting a categorical label) and regression (predicting a continuous value) are common supervised learning tasks. This project focuses on a binary classification problem.

A typical supervised machine learning workflow involves:

1. **Data Collection and Preparation:** Gathering and preprocessing data, which includes cleaning, feature scaling, and splitting into training and testing sets.
2. **Model Selection:** Choosing an appropriate model architecture (e.g., logistic regression, support vector machines, neural networks) capable of learning the underlying patterns in the data.
3. **Loss Function Definition:** Selecting a loss function (or cost function) that quantifies the discrepancy between the model's predictions and the true target values. For classification, cross-entropy loss is a common choice.
4. **Optimization:** Training the model by iteratively adjusting its parameters to minimize the loss function. This is typically achieved using optimization algorithms like gradient descent or its variants.
5. **Evaluation:** Assessing the trained model's performance on unseen data (the test set) using appropriate metrics (e.g., accuracy, precision, recall).

Quantum machine learning aims to leverage quantum computational principles to potentially enhance or offer new approaches to these classical machine learning tasks.

### 2.2.2 Variational Quantum Algorithms

Variational quantum algorithms form the foundation of near-term quantum machine learning. These algorithms combine:

1. **State preparation:** Encoding classical data into quantum states
2. **Parameterized quantum circuit:** Processing the encoded data
3. **Measurement:** Extracting classical information from the quantum state
4. **Classical optimization:** Updating circuit parameters based on measurement results

**Data encoding** is a crucial step, mapping classical data $\mathbf{x}$ into a quantum state $|\phi(\mathbf{x})\rangle$. One common method is **angle encoding**, where classical features $\mathbf{x} = (x_1, \ldots, x_d)$ are encoded into the rotation angles of single-qubit gates. For instance, each feature $x_i$ can be mapped to a rotation on qubit $i$:

$$|\phi(\mathbf{x})\rangle = \bigotimes_{i=1}^{d} \left( \cos\left(\frac{\pi x_i}{2}\right) |0\rangle + \sin\left(\frac{\pi x_i}{2}\right) |1\rangle \right) \tag{2.10}$$

This can be implemented by applying a Hadamard gate followed by a $R_z$ rotation gate to each qubit:

$$U_{\text{enc}}(\mathbf{x}) = \prod_{i=1}^{d} R_z(2\pi x_i) H_i \tag{2.11}$$

More complex encoding strategies, like the **ZZ feature map**, can capture interactions between features by introducing entangling gates. The ZZ feature map is defined as:

$$U_\Phi(\mathbf{x}) = \exp\left( i \sum_j \phi_j(\mathbf{x}) Z_j \right) \exp\left( i \sum_{j,k} \phi_{j,k}(\mathbf{x}) Z_j Z_k \right) \left( \bigotimes H \right) \tag{2.12}$$

where $\phi_j(\mathbf{x})$ and $\phi_{j,k}(\mathbf{x})$ are functions of the input features, often simple linear combinations (e.g., $\phi_j(\mathbf{x}) = x_j$ and $\phi_{j,k}(\mathbf{x}) = (\pi - x_j)(\pi - x_k)$). This encoding creates entanglement between qubits, potentially capturing non-linear relationships in the data.

Once the data is encoded, a **parameterized quantum circuit (PQC)**, also known as an ansatz, processes the quantum state. The PQC consists of a sequence of fixed and parameterized quantum gates. The overall unitary transformation applied by the PQC can be written as:

$$U(\boldsymbol{\theta}) = \prod_{l=1}^{L} U_l(\boldsymbol{\theta}_l) U_{\text{ent}} \tag{2.13}$$

where $U_l(\boldsymbol{\theta}_l)$ represents a layer of parameterized single-qubit rotations (e.g., $R_y(\theta)$ or $R_z(\theta)$ gates) with parameters $\boldsymbol{\theta}_l$, and $U_{\text{ent}}$ represents a layer of entangling gates (e.g., CNOT gates). The structure of $U(\boldsymbol{\theta})$ is often repeated for $L$ layers to increase the expressivity of the model. The choice of ansatz architecture, including the types of gates and their arrangement, is critical and problem-dependent.

## 2.3 Optimization and Training

The goal of training a quantum machine learning model is to find the optimal set of parameters $\boldsymbol{\theta}$ for the parameterized quantum circuit (PQC) that minimizes a given loss function. This process typically involves iteratively updating the parameters based on the gradient of the loss function with respect to these parameters.

For binary classification tasks, a common choice for the loss function is the cross-entropy loss. It measures the dissimilarity between the true labels $y_i$ and the model's predictions $f(\mathbf{x}_i; \boldsymbol{\theta})$ for a dataset of $N$ samples:

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{N} \sum_{i=1}^{N} \left[ y_i \log f(\mathbf{x}_i; \boldsymbol{\theta}) + (1 - y_i) \log(1 - f(\mathbf{x}_i; \boldsymbol{\theta})) \right] \tag{2.14}$$

Here, $f(\mathbf{x}_i; \boldsymbol{\theta})$ represents the probability predicted by the quantum model for sample $\mathbf{x}_i$ given parameters $\boldsymbol{\theta}$.

To minimize this loss function, gradient-based optimization algorithms are employed. The fundamental step in these algorithms is the computation of the gradient $\nabla_{\boldsymbol{\theta}} \mathcal{L}$. For quantum circuits, the gradient of an expectation value $\langle \hat{O} \rangle = \langle \psi(\boldsymbol{\theta}) | \hat{O} | \psi(\boldsymbol{\theta}) \rangle$ with respect to a gate parameter $\theta_j$ can be calculated exactly using the parameter-shift rule:

$$\frac{\partial}{\partial \theta_j} \langle \psi(\boldsymbol{\theta}) | \hat{O} | \psi(\boldsymbol{\theta}) \rangle = \frac{1}{2} \left[ \langle \psi(\boldsymbol{\theta}^+) | \hat{O} | \psi(\boldsymbol{\theta}^+) \rangle - \langle \psi(\boldsymbol{\theta}^-) | \hat{O} | \psi(\boldsymbol{\theta}^-) \rangle \right] \tag{2.15}$$

where $\boldsymbol{\theta}_j^{\pm} = \boldsymbol{\theta} \pm \frac{\pi}{2} \mathbf{e}_j$, with $\mathbf{e}_j$ being a unit vector with a 1 at the $j$-th position and zeros elsewhere. This rule allows for the analytical computation of gradients by evaluating the circuit twice with shifted parameters.

The simplest gradient-based optimization algorithm is gradient descent, where parameters are updated in the direction opposite to the gradient: $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}_t$, where $\eta$ is the learning rate. While straightforward, gradient descent can suffer from slow convergence or getting trapped in local minima.

More advanced optimizers, such as Adam (Adaptive Moment Estimation), often provide better performance. Adam combines the advantages of two other extensions of stochastic gradient descent: AdaGrad, which handles sparse gradients well, and RMSProp, which works well in online and non-stationary settings. Adam computes

adaptive learning rates for each parameter by estimating first and second moments of the gradients. The update rules for Adam at iteration $t$ are:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \nabla_{\boldsymbol{\theta}} \mathcal{L}_t \tag{2.16}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) (\nabla_{\boldsymbol{\theta}} \mathcal{L}_t)^2 \tag{2.17}$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t} \tag{2.18}$$

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t} \tag{2.19}$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \tag{2.20}$$

Here, $\mathbf{m}_t$ and $\mathbf{v}_t$ are estimates of the first moment (the mean) and the second moment (the uncentered variance) of the gradients, respectively. $\beta_1$ and $\beta_2$ are exponential decay rates for these moment estimates (typically close to 1). $\hat{\mathbf{m}}_t$ and $\hat{\mathbf{v}}_t$ are bias-corrected estimates. $\alpha$ is the step size (learning rate), and $\epsilon$ is a small constant added for numerical stability (e.g., $10^{-8}$). Adam often converges faster and more reliably than standard gradient descent, making it a popular choice for training machine learning models, including VQAs.

# Chapter 3

# Implementation

This chapter details the implementation of our quantum machine learning framework, including data preprocessing, quantum circuit design, and optimization procedures.

## 3.1 Dataset and Preprocessing

We use the classical Iris dataset for binary classification, focusing on the first two classes: Iris Setosa (class 0) and Iris Versicolor (class 1). The dataset contains 100 samples with 4 features each: For computational efficiency, we limit our analysis to the first 2 features,

| **Feature Index** | **Feature Name** |
|---|---|
| 1 | Sepal length (cm) |
| 2 | Sepal width (cm) |
| 3 | Petal length (cm) |
| 4 | Petal width (cm) |

Table 3.1: Features in the Iris dataset used for quantum machine learning classification.

requiring only 2 qubits for encoding.

The preprocessing pipeline consists of: The normalization ensures that feature values

| **Step** | **Preprocessing Operation** |
|---|---|
| 1 | Feature selection: Extract the first $n$ features where $n$ equals the number of qubits |
| 2 | Train-test split: 70% training, 30% testing with stratified sampling |
| 3 | Standardization: Apply z-score normalization using sklearn's StandardScaler |
| 4 | Range mapping: Map features to $[0, 1]$ interval for quantum gate parameters |

Table 3.2: Data preprocessing pipeline for quantum machine learning implementation.

are suitable for quantum rotation angles, mapping scaled features $x_{\text{scaled}}$ to $x_{\text{norm}}$:

$$x_{\text{norm}} = \frac{x_{\text{scaled}} - x_{\text{scaled}}^{\min}}{x_{\text{scaled}}^{\max} - x_{\text{scaled}}^{\min}} \tag{3.1}$$

## 3.2 Quantum Circuit Architecture

Our quantum classifier follows the standard variational quantum algorithm structure. The quantum circuits are designed and simulated using the Qiskit library [2]. The

total unitary operation $U_{\text{total}}$ is a composition of encoding, ansatz, and measurement operations:

$$U_{\text{total}} = U_{\text{measurement}} \circ U_{\text{ansatz}}(\boldsymbol{\theta}) \circ U_{\text{encoding}}(\mathbf{x}) \tag{3.2}$$

where $U_{\text{encoding}}(\mathbf{x})$ is the data encoding circuit, $U_{\text{ansatz}}(\boldsymbol{\theta})$ is the parameterized ansatz circuit, and $U_{\text{measurement}}$ represents the measurement in the computational basis.

Two primary data encoding circuits are explored. The first is a **simple angle** encoder. This encoder applies Hadamard gates to each qubit to create a superposition, followed by RZ rotations parameterized by the scaled input features $x_i$. This creates the quantum state:

$$|\psi_{\text{simple}}(\mathbf{x})\rangle = \bigotimes_{i=1}^{d} HR_y(2\pi x_i)|0\rangle \tag{3.3}$$

where $d$ is the number of features (qubits). A visual representation of the simple angle encoder circuit is shown in Figure 3.1.

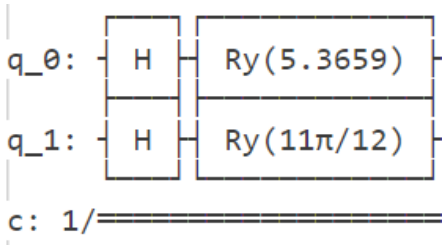


Figure 3.1: Circuit diagram for the Simple Angle Encoder. Each feature $x_i$ is encoded onto a qubit using a Hadamard gate followed by an RY rotation. The numbers represents the first sample features $2\pi$ scaled

The second encoding method is a **ZZ feature map encoder** Inspired by the paper [3]. This advanced encoder implements a ZZ feature map which includes both linear terms (single-qubit RZ rotations based on features $x_i$) and quadratic terms (two-qubit interactions involving CNOT gates and RZ rotations based on products of features $x_i x_j$). The resulting quantum state incorporates feature correlations:

$$|\psi_{\text{ZZ}}(\mathbf{x})\rangle = \left(\prod_{i<j} \text{CNOT}_{i,j} R_z(2\pi x_i x_j) \text{CNOT}_{i,j}\right) \left(\prod_{k=1}^{d} R_z(2\pi x_k) H_k\right) |0\rangle^{\otimes d} \tag{3.4}$$

The circuit for the ZZ feature map encoder is depicted in Figure 3.2.

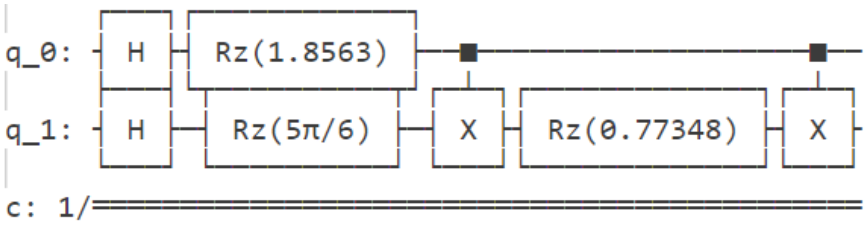


Figure 3.2: Circuit diagram for the ZZ Feature Map Encoder, showing Hadamard followed rotations and two-qubit entangling operations based on feature interactions.The angles represent the firstsecond last sample features $2\pi$ scaled.

Following data encoding, a parameterized ansatz circuit processes the quantum state. We investigate two ansatz structures. The **simple ansatz** applies a layer of single-qubit

RY rotations, followed by a layer of entangling CNOT gates (linear entanglement), and concludes with another layer of single-qubit RY rotations. This ansatz requires $2n$ parameters, where $n$ is the number of qubits. Figure 3.3 illustrates this ansatz.

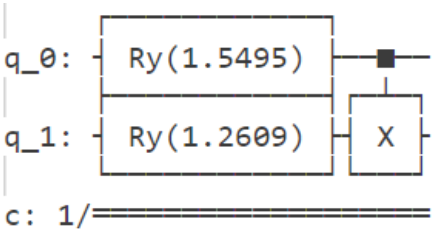


Figure 3.3: Circuit diagram for the Simple Ansatz, consisting of RY rotations and CNOT gates. The total number of parameters is 2. The angle numbers represent random numbers.

The **advanced ansatz** is more complex also Inspired by [3], including multiple layers. Each layer consists of RY rotations, followed by RZ rotations on all qubits, and then a layer of entangling CNOT gates arranged in a circular topology (e.g., qubit $i$ controls qubit $(i + 1) \pmod n$). This structure requires $2n \times L$ parameters, where $L$ is the number of layers and $n$ is the number of qubits. The advanced ansatz circuit is shown in Figure 3.4.

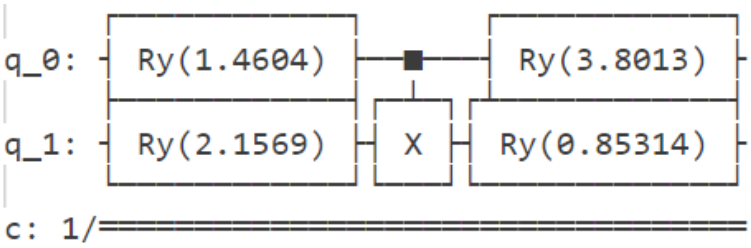


Figure 3.4: Circuit diagram for the Advanced Ansatz, two layer of RY with one CNOT in between. Parameter count is 4. The angle numbers represent random numbers

A complete configurationn of an example quantum circuit with measurement is shown in Figure 3.5.

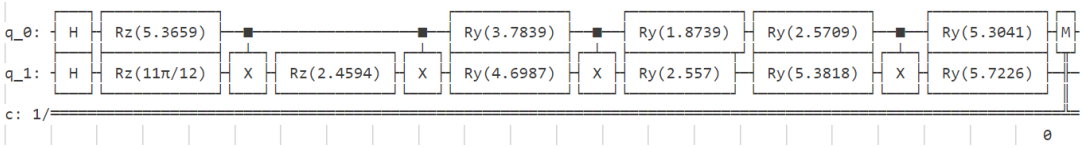


Figure 3.5: This circuit diagram is prepared with ZZ feature map encoder, two layers of advanced ansatz and a measurement operation on qbit 0. Total parameters count is 8.

## 3.3 Training Procedure

The training of the quantum model involves parameter initialization, gradient computation, and optimization. Model parameters $\theta_i$ are initialized uniformly at random in the range $[0, 2\pi]$:

$$\theta_i \sim \mathcal{U}(0, 2\pi) \quad \forall i \tag{3.5}$$

Analytical gradient computation is performed using the parameter-shift rule. For a model output $f(x_i; \boldsymbol{\theta})$ that depends on parameters $\boldsymbol{\theta}$, the derivative with respect to a parameter $\theta_j$ is given by:

$$\frac{\partial f(x_i; \boldsymbol{\theta})}{\partial \theta_j} = \frac{f(x_i; \dots, \theta_j + \pi/2, \dots) - f(x_i; \dots, \theta_j - \pi/2, \dots)}{2} \tag{3.6}$$

This analytical gradient of the model output is then used to compute the gradient of the cross-entropy loss function $L$ with respect to the parameters $\boldsymbol{\theta}_k$:

$$\frac{\partial L}{\partial \boldsymbol{\theta}_k} = \sum_{i=1}^{N} \frac{f_i - y_i}{f_i(1 - f_i)} \frac{\partial f_i}{\partial \boldsymbol{\theta}_k} \tag{3.7}$$

where $N$ is the number of samples, $y_i$ are the true labels, and $f_i = f(x_i; \boldsymbol{\theta})$ is the model's prediction for sample $x_i$.

For optimization, we employ the Adam optimizer. The hyperparameters for Adam are set as follows:

| **Parameter** | **Value** |
|---|---|
| Learning rate ($\alpha$) | [0.1, 0.05, 0.01] |
| Momentum decay ($\beta_1$) | 0.9 |
| RMSprop decay ($\beta_2$) | 0.999 |
| Epsilon ($\epsilon$) | $10^{-8}$ |

Table 3.3: Adam optimizer hyperparameters used for quantum machine learning model training.

### 3.3.1 Batch Processing

To improve training efficiency and generalization, especially for larger datasets, mini-batch gradient descent is employed. Instead of computing the gradient based on the entire training dataset at each step, the data is divided into smaller batches. The training process iterates through epochs, and within each epoch, the training data is typically shuffled to introduce randomness and prevent the model from learning the order of the data. The model parameters are updated after processing each mini-batch.

While Equation 3.7 defines the gradient computation for the full dataset, in mini-batch gradient descent, the gradient is computed for each mini-batch $B$:

$$\nabla_{\boldsymbol{\theta}} L_B = \frac{1}{|B|} \sum_{i \in B} \nabla_{\boldsymbol{\theta}} \ell(f(x_i; \boldsymbol{\theta}), y_i) \tag{3.8}$$

where $\ell$ is the per-sample loss (e.g., binary cross-entropy for a single sample), and $|B|$ is the batch size. The parameter update rule using an optimizer like Adam then uses this batch gradient $\nabla_{\boldsymbol{\theta}} L_B$.

## 3.4 Measurement and Prediction

To generate a prediction from the quantum circuit, we measure the first qubit in the computational basis. The prediction $p(y = 1|\mathbf{x})$ is the probability of measuring this qubit in the $|1\rangle$ state:

$$p(y = 1|\mathbf{x}) = \langle \psi(\mathbf{x}, \boldsymbol{\theta}) | M_0 | \psi(\mathbf{x}, \boldsymbol{\theta}) \rangle \tag{3.9}$$

where $M_0 = |1\rangle\langle 1|_{q_0} \otimes I^{\otimes(n-1)}$ is the measurement operator acting on the first qubit (indexed as $q_0$) and $I$ is the identity on the other $n-1$ qubits. For binary classification, a decision threshold of 0.5 is applied to this probability to obtain the predicted class label $\hat{y}$:

$$\hat{y} = \begin{cases} 1 & \text{if } p(y=1|\mathbf{x}) \geq 0.5 \\ 0 & \text{otherwise} \end{cases} \tag{3.10}$$

## 3.5 Experimental Setup

The experimental setup, including simulation parameters, model configurations, and performance metrics, is summarized in Tables 3.4, 3.5, and 3.6.

| **Parameter Category** | **Details** |
|---|---|
| Quantum backend | Qiskit Aer QASM simulator |
| Shots per measurement | 30 |
| Training epochs | 50 |
| Batch size | [8, 16, 32] |
| Learning rates | [0.1, 0.05, 0.01] |
| Number of ansatz layers | [1, 2, 3] |
| Random seed | 42 (for reproducibility) |

Table 3.4: Simulation parameters for the quantum machine learning experiments.

| **Configuration ID** | **Description** |
|---|---|
| 1 | Simple encoder + Simple ansatz |
| 2 | Simple encoder + Advanced ansatz |
| 3 | ZZ encoder + Simple ansatz |
| 4 | ZZ encoder + Advanced ansatz |

Table 3.5: Evaluated model configurations, combining different encoders and ansatzes.

| **Metric** | **Description** |
|---|---|
| Accuracy | $\frac{\text{Correct predictions}}{\text{Total predictions}}$ |
| Cross-entropy loss | Training convergence metric |
| Training time | Computational efficiency measure |
| Circuit depth | Quantum resource requirement |

Table 3.6: Performance metrics used for model evaluation.

# Chapter 4

# Results

This chapter presents the empirical results obtained from training and evaluating the various quantum machine learning models described in Chapter 3. We analyze the impact of different hyperparameters and architectural choices, such as learning rates, batch sizes, number of ansatz layers, and the complexity of encoders and ansatzes, on the validation accuracy. All models were trained for 50 epochs, and the validation accuracy was recorded at each epoch. The experiments were repeated multiple times with different random initializations, and the plots show these individual runs to illustrate the variability and general trends. From here on the encoder type "ZZ Feature Map Encoder" referred earlier will be referred to as "advanced encoder" and "HadRotZZEncoder" becasue it is the name of the class in the codebase and testing.

## 4.1 Effect of Learning Rate

Figure 4.1 illustrates the validation accuracy over 50 epochs for different learning rates (0.01, 0.05, and 0.1) across four distinct model configurations: Simple Encoder with Simple Ansatz, Simple Encoder with Advanced Ansatz, Advanced Encoder with Simple Ansatz, and Advanced Encoder with Advanced Ansatz.

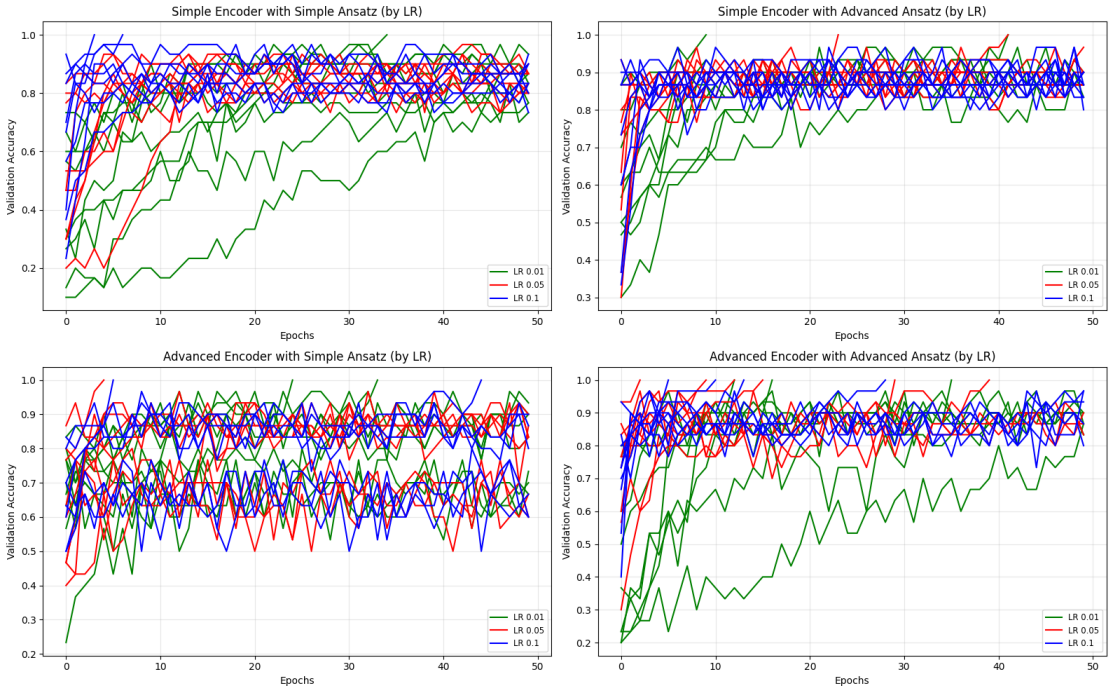


Figure 4.1: Validation accuracy versus epochs for different learning rates (LR). Each subplot represents a different encoder-ansatz combination: (Top-Left) Simple Encoder + Simple Ansatz, (Top-Right) Simple Encoder + Advanced Ansatz, (Bottom-Left) Advanced Encoder + Simple Ansatz, (Bottom-Right) Advanced Encoder + Advanced Ansatz. Learning rates of 0.01 (green), 0.05 (red), and 0.1 (blue) are compared.

Across all configurations, a learning rate of 0.1 (blue lines) generally leads to faster initial convergence but also exhibits high variance around 0.9 accuracy in later epochs, For instance, in the "Simple Encoder with Advanced Ansatz" configuration, the 0.1 LR runs clearly show rapid initial improvement but then fluctuate around 0.9 accuracy. The typical initial convergence is occures between 1 and 3 epochs, which is very quick.

A learning rate of 0.05 (red lines) appears to have similar convergence as 0.1 (blue). We dont see any clear stability signs, but generally, lower learning rates lead to more stable training curves.

The smallest learning rate, 0.01 (green lines), generally shows the slowest convergence and often appear to reach convergence at around 50 epochs. While it can lead to stable training and good final accuracies, particularly for more complex models like "Advanced Encoder with Advanced Ansatz", it may require more epochs to reach its peak performance compared to the 0.05 LR. In some cases, such as "Simple Encoder with Simple Ansatz", the 0.01 LR struggles to achieve accuracies comparable to higher learning rates within the 50-epoch timeframe.

Overall, a learning rate of 0.05 seems to be the most robust choice for these models and datasets, providing a good trade-off between learning speed and potential stability.

We do notice that in the "Advanced Encoder with Simple Ansatz" configuration, the The accuracies curves seems to de devided into two groups of accuracies convergence, we will discuss this phenomenon later.

It should be noted that even though we look at one paramter variable at a time the other parameters are do vary as well, so the results are not fully independent. However, we can still draw some insights from looking at the results like this.

## 4.2 Effect of Ansatz Layers

The impact of the number of layers in the ansatz (1, 2, or 3 layers) on validation accuracy is depicted in Figure 4.2.

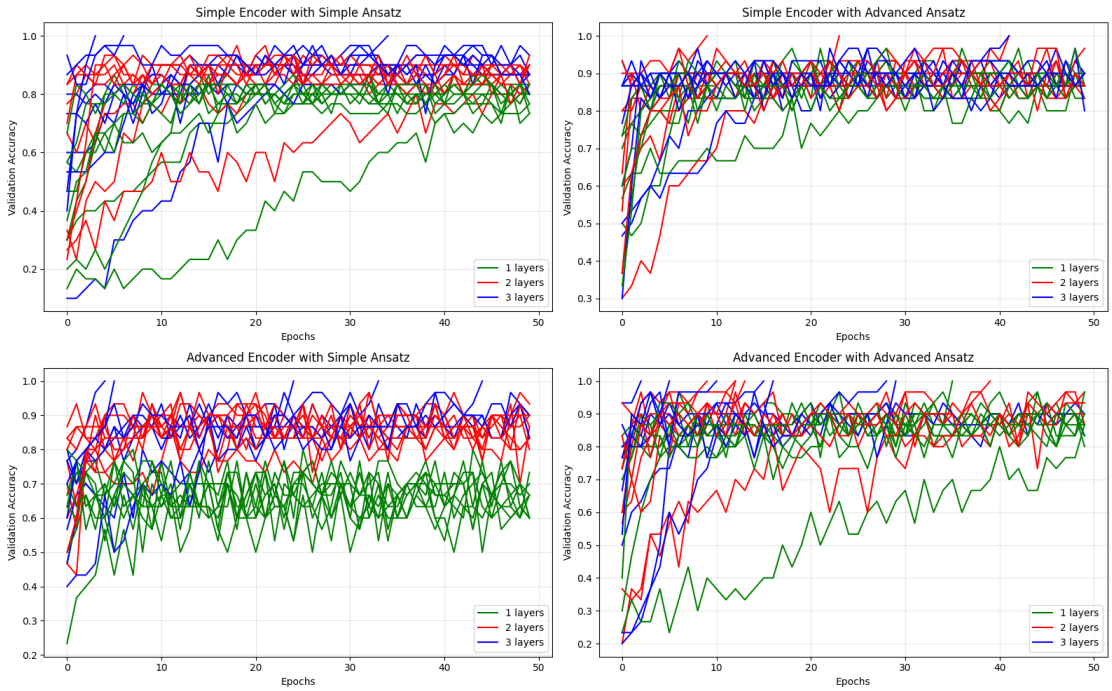


Figure 4.2: Validation accuracy versus epochs for different numbers of ansatz layers. Each subplot represents a different encoder-ansatz combination: (Top-Left) Simple Encoder + Simple Ansatz, (Top-Right) Simple Encoder + Advanced Ansatz, (Bottom-Left) Advanced Encoder + Simple Ansatz, (Bottom-Right) Advanced Encoder + Advanced Ansatz. Models with 1 layer (green), 2 layers (red), and 3 layers (blue) are compared.

Looking at at the configurations with the "Simple Encoder with Simple Ansatz" and "Advanced Encoder with Simple Ansatz," we clearly see that one layer (green) is not able to achieve the same level of accuracy as two layers (red) or three layers (blue). This suggests that for simpler ansatzes, there is not enough parameters to capture the complexity of the data. This insight is imidiately visible in the "Advanced Encoder with Simple Ansatz" configuration, where there is a clear separation between the accuracies of the 1 layer (green) and the 2 and 3 layers (red and blue). The 1 layer model struggles to get higher then 0.8 accuracy, while the 2 and 3 layers models achieve accuracies above 0.9.

Having two layers with parameter count being 4 in the simple ansatzes seems outperforming advanced ansatzes with one layer, in terms of quick convergence. Even though one layer advanced ansatzes is able to reach high accuracy, it takes longer to converge compared to the two layers simple ansatzes.

Other insight we get is that it dont seem to be significant difference between two layers (red) and three layers (blue) in terms accuracies and convergence speed. Although there seem to be a slightly faster convergence for the three layers (blue) models. Three layers ansatzes do have more parameters that needs to be optimized, that can takes longer computation time, but in terms of epochs 3 layers convergence are faster.

Another key observed is that for complex enough configurations The model is able to

reach 100% accuracy on the validation set. These are appearnt as top spikes reaching 1.0 in the plots, the model then stops the optimization process and the training is stopped.

## 4.3 Effect of Batch Size

Figure 4.3 shows the validation accuracy for different batch sizes (8, 16, and 32) for two encoder types ("AngleEncoder" and "HadRotZZEncoder") combined with both Simple and Advanced Ansatzes.

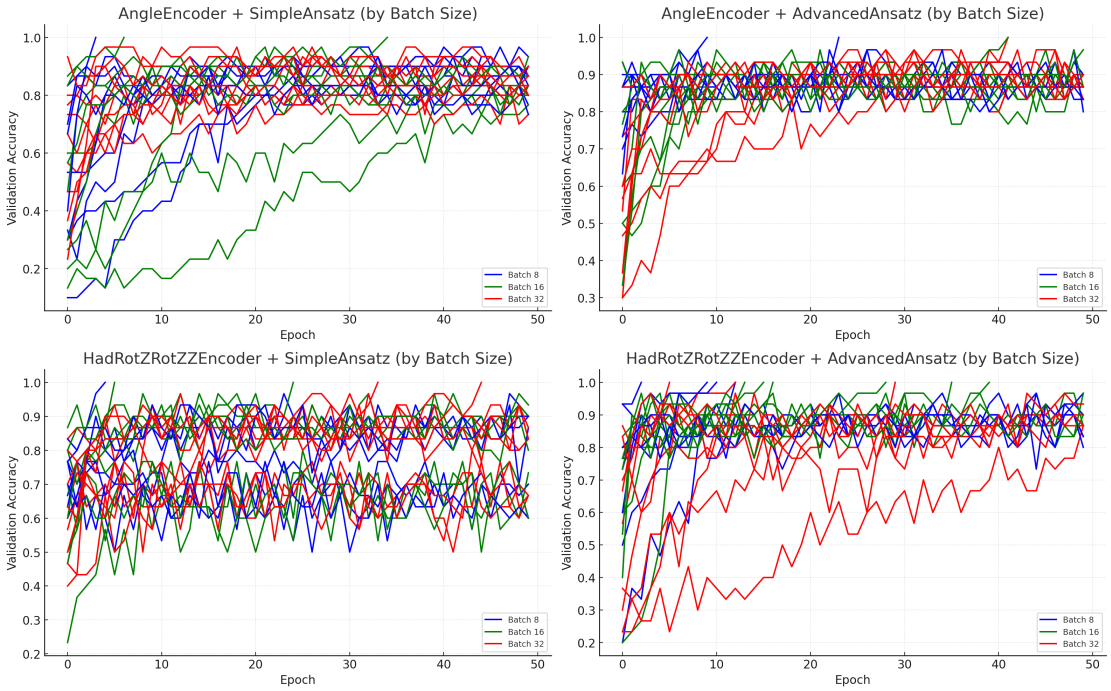


Figure 4.3: Validation accuracy versus epochs for different batch sizes. The top row uses an "AngleEncoder" with (Left) Simple Ansatz and (Right) Advanced Ansatz. The bottom row uses a "HadRotZZEncoder" with (Left) Simple Ansatz and (Right) Advanced Ansatz. Batch sizes of 8 (green), 16 (red), and 32 (blue) are compared.

It is harder to draw conclusions from the batch size results comparisions. Since we cannot see the same clear separation between the batch sizes it is suggested to run the model with a larger batch size forexample 32 since larger batch takes less time to compute. In general, smaller batch sizes can lead to better generalization and higher validation accuracies.

## 4.4 Comparison of Encoder and Ansatz Architectures

Figure 4.4 directly compares the performance of the Simple Ansatz versus the Advanced Ansatz for two different encoder types: "Angle Encoder" and "Advanced Encoder" (presumably the ZZ Feature Map Encoder).

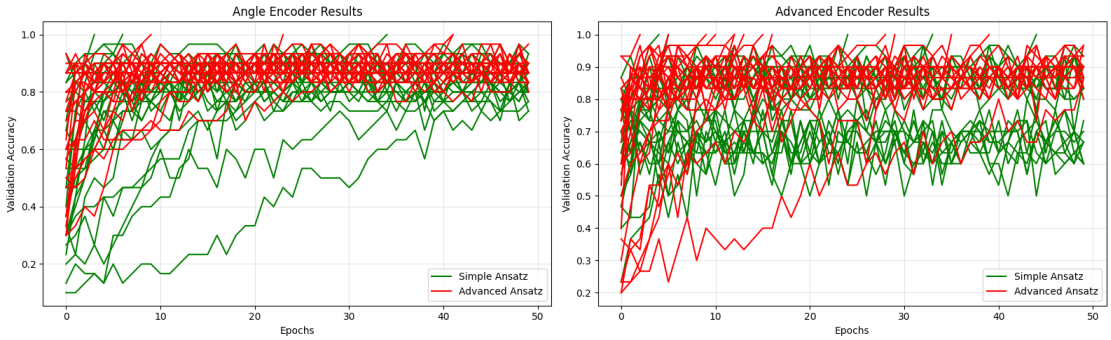


Figure 4.4: Validation accuracy versus epochs comparing Simple Ansatz (green) and Advanced Ansatz (red). (Left) Results for the Angle Encoder. (Right) Results for the Advanced Encoder.

For the "Angle Encoder" (left panel), the Advanced Ansatz (red lines) most of the times outperforms the Simple Ansatz, it is especially appearent in the "Simple Encoder" results. Where "Advanced Ansatz" consistently achieves high validation accuracies around 0.9.

## 4.5 Visualization of Model Decision Boundaries

To further understand the behavior of different model architectures, we visualize the decision boundaries learned by two contrasting configurations: a Simple Encoder with a Simple Ansatz (1 layer) and an Advanced Encoder with an Advanced Ansatz (1 layer). Both models were trained using the same hyperparameters for learning rate and batch size. The decision boundary plots illustrate the model's predicted probability for Class 1 across a 2D feature space (using the first two features of the Iris dataset for visualization).

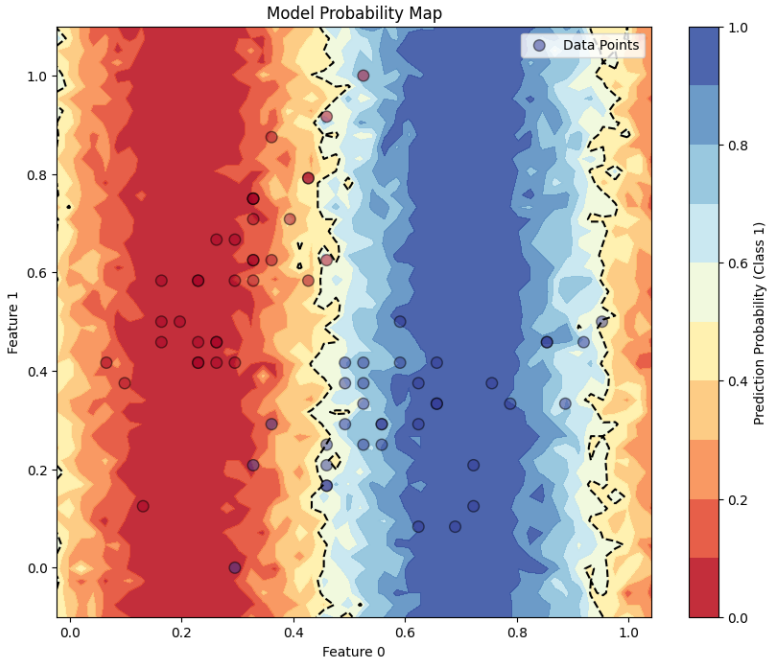


Figure 4.5: Model probability map and decision boundary for the Simple Encoder with Simple Ansatz (1 layer). The dashed line represents the $P(\text{Class } 1) = 0.5$ contour. Data points for the two classes are overlaid. This model achieved a validation accuracy of 0.8.

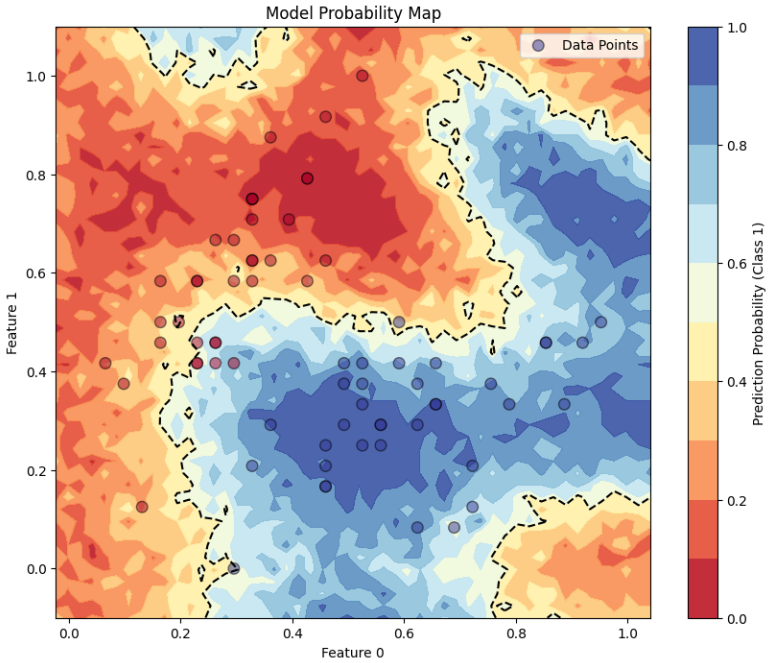


Figure 4.6: Model probability map and decision boundary for the Advanced Encoder with Advanced Ansatz (1 layer). The dashed line represents the $P(\text{Class } 1) = 0.5$ contour. Data points for the two classes are overlaid. This model achieved a validation accuracy of 0.9.

Figure 4.5 displays the decision boundary for the Simple Encoder combined with a Simple Ansatz (1 layer). The boundary appears relatively smooth and carves out broader regions for each class. While it manages to separate a majority of the data points, its simplicity limits its ability to capture more intricate patterns in the feature space, consistent with its validation accuracy of 0.8. The transition between class probabilities is somewhat gradual.

In contrast, Figure 4.6 shows the decision boundary for the Advanced Encoder paired with an Advanced Ansatz (1 layer). This model, which achieved a higher validation accuracy of 0.9, exhibits a more complex and flexible decision boundary. It is capable of forming more nuanced regions and appears to more closely follow the distribution of the data points, leading to a better separation of the two classes. The increased complexity of both the encoder and the ansatz allows the model to learn a more expressive mapping from features to predictions.

Comparing the two figures, the advanced architecture (Figure 4.6) demonstrates a clear advantage in its ability to model the underlying data distribution more effectively. The more convoluted boundary suggests a higher capacity to fit the training data, which, in this case, translates to better generalization on the validation set. This visual evidence complements the quantitative accuracy metrics, illustrating how increased model complexity can lead to improved classification performance by enabling the learning of more sophisticated decision surfaces.

## 4.6 Summary of Results

The experiments highlighted the significant impact of hyperparameter choices and circuit architecture on the performance of quantum machine learning models for the Iris dataset.

- **Learning Rate:** A learning rate of 0.05 offered a good balance between convergence speed and stability, while 0.1 led to faster but more variable convergence, and 0.01 was often too slow within 50 epochs. The optimal rate is model-dependent, and peculiar convergence patterns in some configurations warrant further study.
- **Ansatz Layers:** Increasing ansatz layers from one to two generally improved model expressiveness and accuracy, especially for simpler ansatz structures. Gains diminished when moving from two to three layers, suggesting a trade-off with computational cost.
- **Batch Size:** No single batch size proved universally optimal. While larger batches (e.g., 32) are computationally efficient, the expected generalization benefit of smaller batches was not clearly observed in these trials.
- **Encoder and Ansatz Complexity:** More complex architectures, like the "Advanced Ansatz" and "Advanced Encoder," generally yielded higher validation accuracies, demonstrating the value of greater model expressiveness.
- **Overall Observations and Future Directions:** Hyperparameter tuning and architectural design are crucial. The observed variability in results underscores the stochastic nature of training. Future work should incorporate k-fold cross-validation for more robust evaluations. Further investigations could also explore different optimizers, the impact of shot noise, analysis of training loss, and scalability to more complex datasets.

In essence, while quantum models demonstrate potential, achieving optimal performance requires careful configuration. More expressive models tend to perform better but necessitate more thorough tuning and greater computational resources.

# Chapter 5

# Conclusion

This project successfully implemented and evaluated a quantum machine learning (QML) model for binary classification of the Iris dataset. We systematically investigated the impact of various architectural choices and hyperparameters, including data encoders, parameterized ansatzes, learning rates, ansatz layers, and batch sizes, on model performance.

## 5.1 Summary of Key Findings

Our empirical results, detailed in Chapter 4, highlight several key takeaways:

- **Architectural Complexity Matters:** More expressive quantum circuits, achieved through advanced encoders (e.g., ZZ Feature Map) and more complex ansatz structures, generally yielded higher validation accuracies. This was visually corroborated by the decision boundary plots, where advanced models demonstrated a capacity to learn more nuanced data separations. For instance, the combination of an Advanced Encoder with an Advanced Ansatz consistently performed well, achieving validation accuracies around 0.9.
- **Hyperparameter Sensitivity:** The performance of the QML models was notably sensitive to hyperparameter settings. A learning rate of 0.05 generally provided a good balance between convergence speed and stability across different configurations. Increasing the number of ansatz layers from one to two typically improved performance, particularly for simpler ansatz designs, while further increases to three layers offered diminishing returns. The effect of batch size was less conclusive, suggesting that while larger batches can improve computational efficiency, their impact on generalization in this context requires further investigation.
- **Parameter-Shift Rule Efficacy:** The use of the parameter-shift rule for analytical gradient computation, coupled with the Adam optimizer, enabled effective training of the variational quantum circuits.
- **Model Expressiveness vs. Simplicity:** Simpler models, such as a Simple Encoder with a 1-layer Simple Ansatz, while easier to implement and faster to train per epoch, often struggled to achieve the same level of accuracy as their more complex counterparts, sometimes plateauing at lower performance levels (e.g., 0.8 validation accuracy).

- **Variability in Training:** The experiments revealed variability in training outcomes across different runs with identical configurations but different random initializations. This underscores the stochastic nature of the training process for these QML models and the importance of multiple trials for robust assessment.

## 5.2 Discussion and Critical Comments

The methods employed, leveraging parameterized quantum circuits, demonstrate the potential of QML for classification tasks. The ability to encode classical data into high-dimensional Hilbert spaces and process it using quantum operations offers a novel paradigm for machine learning. The primary strength of the approach lies in its potential for quantum advantage, although demonstrating this definitively requires more complex problems and, eventually, fault-tolerant quantum hardware.

A significant challenge observed was the intricate interplay between circuit architecture and hyperparameter settings. Finding an optimal configuration often requires extensive experimentation. While the parameter-shift rule provides an elegant way to compute gradients, the overall training process can still be computationally intensive, especially as the number of qubits, parameters, or simulation shots increases. The limited number of shots (30 per measurement) used in this study to manage simulation time likely introduced statistical noise, which could affect the precision of gradient estimates and the stability of training.

The choice of encoding strategy and ansatz design is crucial. While the "Advanced Encoder" and "Advanced Ansatz" showed promise, their designs were inspired by existing literature and further tailoring to specific dataset characteristics could yield improvements. The current study focused on a relatively small dataset with few features; the scalability and performance on larger, more complex datasets remain open questions.

## 5.3 Future Directions and Improvements

Based on the findings and limitations of this work, several avenues for future research and improvement can be identified:

- **Robust Evaluation:** Implementing k-fold cross-validation would provide a more robust estimate of model generalization performance and reduce the impact of specific train-test splits.

- **Optimizer Exploration:** While Adam was used, exploring other classical optimizers (e.g., SPSA, L-BFGS) or quantum-aware optimizers could lead to better convergence or final performance.

- **Noise Analysis and Mitigation:** A systematic study of the impact of shot noise by varying the number of shots per measurement would be beneficial. Furthermore, if transitioning to real quantum hardware, exploring noise mitigation techniques will be essential.

- **Advanced Architectures:** Further exploration of more sophisticated data encoding schemes and ansatz designs, potentially leveraging automated or adaptive circuit generation techniques, could unlock better performance. Investigating the entanglement patterns and their impact on expressivity would also be valuable.

- **Scalability Studies:** Testing the developed QML framework on more complex and higher-dimensional datasets, such as the Breast Cancer dataset mentioned in the project description, would provide insights into its scalability and practical applicability.
- **Hybrid Classical-Quantum Models:** Exploring hybrid approaches that combine the strengths of classical neural networks with quantum layers could be a fruitful direction.
- **Theoretical Understanding:** Further theoretical analysis into the expressivity and trainability of the specific quantum circuits used, and their relationship to the classical data's structure, would deepen our understanding.

In conclusion, this project provided a practical exploration of quantum machine learning, demonstrating its capabilities and highlighting areas for future development. The journey from data encoding to model training and evaluation has underscored both the exciting prospects and the current challenges in this rapidly evolving field. The insights gained form a solid foundation for further investigation into the potential of quantum algorithms to solve complex machine learning problems.

## 5.4 Use of Large language models

During the project, GitHub Copilot was utilized as a coding assistant and for learning [4]. Github copilot is a coding assitent developed by github, it offered aid in development process in several ways:

- Code completion suggestions helped reduce boilerplate code and accelerated implementation of common programming patterns
- Generated code snippets served as useful starting points that could be modified and refined to meet specific requirements
- The tool provided helpful context about Python libraries and best practices during development
- Discussion of algorithms and general learning

While Copilot provided valuable assistance, all generated code was carefully reviewed and validated to ensure correctness. The tool was used as an aid to augment human programming capabilities rather than as a complete replacement for manual development. Critical algorithm implementations and core logic were primarily written manually to maintain full understanding and control of the codebase.

The experience demonstrated how AI coding assistants can enhance programmer productivity while still requiring human oversight and domain expertise to produce high-quality, reliable code. This balanced approach allowed us to leverage the benefits of AI assistance while ensuring the implementation met rigorous academic and technical standards.

# Bibliography


[1] Michael A Nielsen and Isaac L Chuang. *Quantum computation and quantum information*. Cambridge University Press, 2010.

[2] IBM Quantum Team. *Qiskit: An Open-source Framework for Quantum Computing*. Version 2.0. 2025. URL: https://qiskit.org/.

[3] Amira Abbas, David Sutter, Christa Zoufal, Aurélien Lucchi, Alessio Figalli and Stefan Woerner. "The power of quantum neural networks". In: *Nature Computational Science* 1.6 (2021), pp. 403–409.

[4] GitHub. *GitHub Copilot · Your AI pair programmer*. https://github.com/features/copilot. Accessed 6th June 2025. 2025.

# Appendix A

# Appendix

This appendix provides additional technical details, code implementations, and supplementary results that support the main findings of this study.

## A.1 Code Implementation Details

### A.1.1 Complete Quantum Classifier Implementation

Listing A.1: Core QuantumClassifier class structure

```
class QuantumClassifier:
    def __init__(self, num_qubits, encoder=None, ansatz=None,
                 optimizer=None, shots=1024):
        self.num_qubits = num_qubits
        self.encoder = encoder or AngleEncoder()
        self.ansatz = ansatz or SimpleAnsatz()
        self.optimizer = optimizer or AdamOptimizer()
        self.shots = shots
        self.backend = AerSimulator()

    def _build_circuit(self, features):
        circuit = QuantumCircuit(self.num_qubits, 1)

        # Encoding
        circuit = self.encoder.encode(circuit, features)

        # Ansatz
        circuit = self.ansatz.apply(circuit, self.parameters)

        # Measurement
        circuit.measure(self.measurement_qubit, 0)
        return circuit

    def predict_single(self, features):
        circuit = self._build_circuit(features)
        transpiled = transpile(circuit, self.backend)
        job = self.backend.run(transpiled, shots=self.shots)
        counts = job.result().get_counts()

        # Calculate probability of measuring |1>
        prob_1 = counts.get('1', 0) / self.shots
```

```
        return prob_1
```

### A.1.2 Parameter-Shift Rule Implementation

Listing A.2: Analytical gradient computation using parameter-shift rule

```
def _compute_gradient_parameter_shift(self, X, y):
    n_params = len(self.parameters)
    gradient = np.zeros(n_params)

    # Compute current loss for efficiency
    current_loss = self.cross_entropy_loss(y, self.predict(X))

    for param_idx in range(n_params):
        # Store original parameter
        original_param = self.parameters[param_idx]

        # Forward shift: theta_i + pi/2
        self.parameters[param_idx] = original_param + np.pi / 2
        y_pred_plus = self.predict(X)
        loss_plus = self.cross_entropy_loss(y, y_pred_plus)

        # Backward shift: theta_i - pi/2
        self.parameters[param_idx] = original_param - np.pi / 2
        y_pred_minus = self.predict(X)
        loss_minus = self.cross_entropy_loss(y, y_pred_minus)

        # Compute gradient: (f(theta+) - f(theta-)) / 2
        gradient[param_idx] = (loss_plus - loss_minus) / 2.0

        # Restore original parameter
        self.parameters[param_idx] = original_param

    return gradient
```

### A.1.3 ZZ Feature Map Detailed Implementation

Listing A.3: Complete ZZ feature map encoder with entangling blocks

```
class HadRotZRotZZEncoder(BaseEncoder):
    def __init__(self, scaling=2 * np.pi):
        self.scaling = scaling

    def encode(self, circuit, features):
        n_qubits = len(features)

        # Step 1: Apply Hadamard gates to create superposition
        for i in range(n_qubits):
            circuit.h(i)

        # Step 2: Single-qubit Z-rotations (linear terms)
        for i in range(n_qubits):
            circuit.rz(self.scaling * features[i], i)
```

```
        # Step 3: Two-qubit ZZ interactions (quadratic terms)
        for i in range(n_qubits):
            for j in range(i + 1, n_qubits):
                # Entangling block: CNOT - RZ - CNOT
                angle = self.scaling * features[i] * features[j]
                circuit.cx(i, j)          # Control: i, Target: j
                circuit.rz(angle, j)      # RZ rotation on target
                circuit.cx(i, j)          # Control: i, Target: j

        return circuit
```

## A.2 Mathematical Derivations

### A.2.1 Cross-Entropy Loss Gradient

For the cross-entropy loss function:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log p_i + (1-y_i)\log(1-p_i)] \tag{A.1}$$

The gradient with respect to model parameters $\theta_j$ is:

$$\frac{\partial \mathcal{L}}{\partial \theta_j} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \frac{1}{p_i}\frac{\partial p_i}{\partial \theta_j} + (1-y_i)\frac{1}{1-p_i}\left(-\frac{\partial p_i}{\partial \theta_j}\right)\right] \tag{A.2}$$

$$= -\frac{1}{N}\sum_{i=1}^{N}\left[\frac{y_i}{p_i} - \frac{1-y_i}{1-p_i}\right]\frac{\partial p_i}{\partial \theta_j} \tag{A.3}$$

$$= -\frac{1}{N}\sum_{i=1}^{N}\frac{y_i - p_i}{p_i(1-p_i)}\frac{\partial p_i}{\partial \theta_j} \tag{A.4}$$

Where $\frac{\partial p_i}{\partial \theta_j}$ is computed using the parameter-shift rule.

### A.2.2 ZZ Feature Map Quantum State

The quantum state prepared by the ZZ feature map can be written as:

$$|\psi(\mathbf{x})\rangle = U_{\mathrm{ZZ}}(\mathbf{x})|0\rangle^{\otimes n} \tag{A.5}$$

where:

$$U_{\mathrm{ZZ}}(\mathbf{x}) = \prod_{i<j}\exp(i\pi x_i x_j Z_i Z_j)\prod_{i=1}^{n}\exp(i\pi x_i Z_i)\prod_{i=1}^{n}H_i \tag{A.6}$$

$$= \prod_{i<j}(\mathrm{CNOT}_{i,j}\cdot R_z(2\pi x_i x_j)\cdot \mathrm{CNOT}_{i,j})\prod_{i=1}^{n}R_z(2\pi x_i)H_i \tag{A.7}$$

This encoding creates entanglement between qubits, enabling the quantum state to capture correlations between input features.